

LAPORAN PRAKTIKUM IF12D
PEMROGRAMAN BERORIENTASI OBJEK
TUGAS PERTEMUAN 4

Disusun untuk Memenuhi Tugas Mata Kuliah Praktikum Pemrograman Berorientasi Objek
Dosen Pengampu:

Mutaqin Akbar, S.Kom., M.T., MCE., MCF.



Disusun oleh :

Nama	: DIMAS NURDIANSYAH
NIM	: 251110058

UNIVERSITAS MERCU BUANA YOGYAKARTA
FAKULTAS TEKNOLOGI INFORMASI
PROGRAM STUDI INFORMATIKA
2025/2026

DAFTAR ISI

DAFTAR ISI	ii
DAFTAR GAMBR	iii
A. Tugas	1
B. Jawaban.....	1
1. Penjelasan Materi	1
2. Source Code	2
1.1. Class Kendaraan	2
1.2. Class Merk	2
1.3. Class Harga	3
1.4. Class Utama.....	3
3. Output.....	4
4. Penjelasan Kode Program	4
4.1. Class Kendaraan	4
4.2. Class Merk	5
4.3. Class Harga	5
4.4. Class Utama.....	6
4.4. Penejelasan flow program	7
5. Penjelasan Cara Runing Program	8

DAFTAR GAMBAR

Gambar 1. 1 Penjelasan Class abstrak	1
Gambar 2. 1 Source code Class Kendaraan	2
Gambar 2. 2 Source code Class Merk.....	2
Gambar 2. 3 Source code Class Harga.....	3
Gambar 2. 4 Source code Class Utama.....	3
Gambar 3. 1 Output Program.....	4
Gambar 4. 1 Struktur Program.....	7

A. Tugas

1. Buatlah kelas abstrak dengan nama 'Kendaraan' yang memiliki kelas konkrit 'Merk' lalu terdapat kelas 'Harga' yang menjadi subclass dari kelas 'Merk', Kemudian jalankan program pada kelas 'Utama' yang berada di luar package kelas lainnya.
2. (Sebagai contoh pada modul, ada kelas dengan nama 'Hewan', lalu memiliki subclass 'HewanBertulangBelakang' dan juga 'HewanTakBertulangBelakang', dari kedua itu memiliki subclass lagi. Dari 'HewanBertulangBelakang' ada 5: Mamalia, Ikan, Burung, Reptil, Amfibi. Lalu dari 'HewanTakBertulangBelakang' ada 'Serangga')
3. Buatlah output untuk menampilkan jenis kendaraan, merk dan Harga yang sesuai. (minimal 3)!
4. Kerjakan tugas sesuai dengan materi yang sudah disampaikan sertakan screenshot source code dan output lalu buatlah dalam laporan berbentuk PDF!

B. Jawaban

1. Penjelasan Materi

Kelas Abstrak adalah kelas yang bersifat sangat umum sehingga tidak dapat diinstansiasi atau dibuat objeknya secara langsung. Kelas abstrak dirancang sebagai kerangka atau *template* yang menyimpan atribut dan metode umum untuk diwariskan kepada subclass-nya. Dalam Java, kelas abstrak diimplementasikan menggunakan keyword `abstract`, seperti berikut:

```
abstract class NamaKelas {  
    abstract void namaMetode();  
}
```

Gambar 1. 1 Penjelasan Class abstrak

Kelas abstrak dapat memiliki metode abstrak, yaitu metode yang hanya dideklarasikan tanpa memiliki isi (*body*). Metode abstrak tersebut wajib diimplementasikan oleh subclass yang mewarisinya. Subclass yang telah mengimplementasikan seluruh metode abstrak disebut kelas konkrit, yaitu kelas yang dapat dibuat objeknya secara langsung.

Hierarki Pewarisan merupakan konsep di mana kelas disusun secara bertingkat dari yang paling umum hingga paling spesifik. Kelas yang berada di tingkat atas disebut superclass, sedangkan kelas yang mewarisi disebut subclass. Subclass mewarisi seluruh atribut dan metode dari superclass-nya menggunakan keyword `extends`. Hierarki pewarisan dapat digambarkan sebagai berikut:

SuperClass (abstract) => SubClass1 (konkrit) => SubClass2

2. Source Code

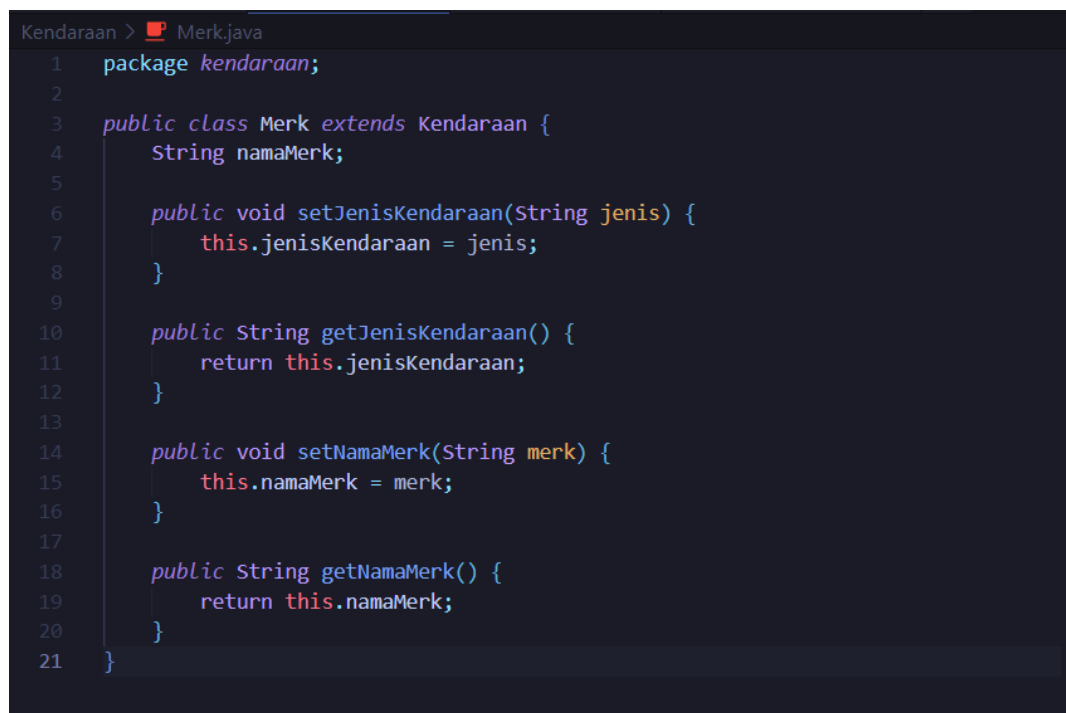
1.1. Class Kendaraan



```
Kendaraan > Kendaraan.java
1 package kendaraan;
2
3 abstract class Kendaraan {
4     String jenisKendaraan;
5     abstract void setJenisKendaraan(String jenis);
6     abstract String getJenisKendaraan();
7 }
```

Gambar 2. 1 Source code Class Kendaraan

1.2. Class Merk



```
Kendaraan > Merk.java
1 package kendaraan;
2
3 public class Merk extends Kendaraan {
4     String namaMerk;
5
6     public void setJenisKendaraan(String jenis) {
7         this.jenisKendaraan = jenis;
8     }
9
10    public String getJenisKendaraan() {
11        return this.jenisKendaraan;
12    }
13
14    public void setNamaMerk(String merk) {
15        this.namaMerk = merk;
16    }
17
18    public String getNamaMerk() {
19        return this.namaMerk;
20    }
21 }
```

Gambar 2. 2 Source code Class Merk

1.3. Class Harga

```
Kendaraan > Harga.java
1  package kendaraan;
2
3  public class Harga extends Merk {
4      private double harga;
5
6      public void setHarga(double harga) {
7          this.harga = harga;
8      }
9
10     public double getHarga() {
11         return this.harga;
12     }
13
14     public String getHargaFormatted() {
15         return String.format("Rp %,0f", this.harga);
16     }
17 }
```

Gambar 2. 3 Source code Class Harga

1.4. Class Utama

```
Utama.java
1  import kendaraan.Harga;
2
3  public class Utama {
4      public static void main(String[] args) {
5
6          Harga[] kendaraan = new Harga[3];
7
8          kendaraan[0] = new Harga();
9          kendaraan[0].setJenisKendaraan("Mobil");
10         kendaraan[0].setNamaMerk("Toyota Avanza");
11         kendaraan[0].setHarga(25000000);
12
13         kendaraan[1] = new Harga();
14         kendaraan[1].setJenisKendaraan("Motor");
15         kendaraan[1].setNamaMerk("Honda CBR");
16         kendaraan[1].setHarga(25000000);
17
18         kendaraan[2] = new Harga();
19         kendaraan[2].setJenisKendaraan("Sepeda");
20         kendaraan[2].setNamaMerk("Polygon");
21         kendaraan[2].setHarga(5000000);
22
23         System.out.println("Nama : Dimas Nurdiansyah");
24         System.out.println("NIM : 251110058");
25         System.out.println();
26
27         System.out.println("=====");
28         System.out.println("      DATA KENDARAAN      ");
29         System.out.println("=====");
30
31         for (int i = 0; i < kendaraan.length; i++) {
32             System.out.println("Kendaraan ke " + (i + 1) + ":");
33             System.out.println("Jenis : " + kendaraan[i].getJenisKendaraan());
34             System.out.println("Merk : " + kendaraan[i].getNamaMerk());
35             System.out.println("Harga : " + kendaraan[i].getHargaFormatted());
36             System.out.println("-----");
37         }
38     }
39 }
40 |
```

Gambar 2. 4 Source code Class Utama

3. Output

```
C:\Users\Bestdym\Data\Praktikum\OOP\Tugas4>javac Kendaraan/*.java Utama.java
C:\Users\Bestdym\Data\Praktikum\OOP\Tugas4>java Utama
Nama : Dimas Nurdiansyah
NIM : 251110058

=====
          DATA KENDARAAN
=====
Kendaraan ke 1:
Jenis : Mobil
Merk : Toyota Avanza
Harga : Rp 250.000.000
-----
Kendaraan ke 2:
Jenis : Motor
Merk : Honda CBR
Harga : Rp 25.000.000
-----
Kendaraan ke 3:
Jenis : Sepeda
Merk : Polygon
Harga : Rp 5.000.000
-----
```

Gambar 3. 1 Output Program

4. Penjelasan Kode Program

4.1. Class Kendaraan

Class Kendaraan adalah kelas abstrak yang berfungsi sebagai blueprint atau setkerangka utama dalam hierarki kelas program ini. Kelas ini tidak dapat dibuat objeknya secara langsung karena bersifat abstract.

Penjelasan Kode Program :

- package kendaraan => menandakan bahwa file ini masuk ke dalam package bernama kendaraan, sehingga dapat diimport oleh kelas lain yang membutuhkannya.
- abstract class Kendaraan => mendeklarasikan bahwa kelas ini bersifat abstrak menggunakan keyword abstract. Artinya kelas ini tidak bisa dibuat objeknya secara langsung dan hanya berfungsi sebagai kerangka untuk kelas-kelas di bawahnya.
- String jenisKendaraan => atribut untuk menyimpan jenis kendaraan seperti Mobil, Motor, atau Sepeda. Atribut ini dapat diakses langsung oleh subclass karena tidak menggunakan akses modifier private.
- abstract void setJenisKendaraan(String jenis) => method abstrak untuk mengisi nilai jenisKendaraan. Karena bersifat abstrak, method ini belum memiliki isi/body dan wajib diimplementasikan oleh subclass yang mewarisinya. Bertipe void karena tidak mengembalikan nilai.
- abstract String getJenisKendaraan() => method abstrak untuk mengambil nilai jenisKendaraan. Wajib diimplementasikan oleh subclass dan bertipe String karena mengembalikan nilai teks jenis kendaraan.

4.2. Class Merk

Class Merk adalah kelas konkrit yang mewarisi class Kendaraan menggunakan keyword `extends`. Karena bersifat konkrit, kelas ini wajib mengimplementasikan seluruh method abstrak yang diwariskan dari Kendaraan.

Penjelasan Kode Program :

- `package kendaraan` => menandakan bahwa file ini berada dalam package yang sama dengan `Kendaraan.java`, sehingga dapat mengakses kelas Kendaraan tanpa perlu melakukan import.
- `public class Merk extends Kendaraan` => mendeklarasikan bahwa kelas Merk mewarisi kelas Kendaraan menggunakan keyword `extends`. Dengan pewarisan ini, Merk secara otomatis memiliki semua atribut dan method dari Kendaraan.
- `String namaMerk` => atribut baru yang ditambahkan di level kelas Merk untuk menyimpan nama merk kendaraan seperti Toyota, Honda, atau Polygon.
- `void setJenisKendaraan(String jenis)` => implementasi dari method abstrak yang diwariskan dari kelas Kendaraan. Di sinilah method tersebut baru mendapatkan isi/body-nya, yaitu mengisi nilai atribut `jenisKendaraan` menggunakan keyword `this` untuk merujuk pada atribut milik kelas ini.
- `String getJenisKendaraan()` => implementasi method abstrak dari Kendaraan untuk mengembalikan nilai `jenisKendaraan` menggunakan perintah `return`.
- `void setNamaMerk(String merk)` => method setter baru yang khusus ditambahkan di kelas Merk untuk mengisi nilai atribut `namaMerk`.
- `String getNamaMerk()` => method getter baru untuk mengambil dan mengembalikan nilai `namaMerk`.

4.3. Class Harga

Class Harga adalah subclass dari Merk yang merupakan kelas paling bawah dalam hierarki. Karena posisinya paling bawah dan tidak lagi bersifat abstrak, dari kelas inilah objek kendaraan sesungguhnya dibuat.

Penjelasan Kode Program :

- `package kendaraan` => menandakan bahwa file ini berada dalam package yang sama sehingga dapat langsung mengakses kelas Merk dan Kendaraan tanpa import.
- `public class Harga extends Merk` => mendeklarasikan bahwa kelas Harga mewarisi kelas Merk. Karena Merk sendiri mewarisi Kendaraan, maka kelas Harga secara tidak langsung juga mewarisi seluruh atribut dan method dari Kendaraan. Hal ini merupakan penerapan multi-level inheritance.
- `private double harga` => atribut baru untuk menyimpan harga kendaraan dalam bentuk angka desimal bertipe `double`. Menggunakan akses modifier `private` sebagai penerapan konsep enkapsulasi, sehingga nilai harga tidak bisa diakses

langsung dari luar kelas dan hanya dapat diakses melalui method getter dan setter.

- `void setHarga(double harga)` => method setter untuk mengisi nilai atribut harga. Parameter menggunakan tipe `double` agar dapat menampung nilai harga yang besar. Keyword `this.harga` digunakan untuk membedakan antara atribut kelas dengan parameter yang memiliki nama sama.
- `double getHarga()` => method getter untuk mengambil nilai harga dalam bentuk angka mentah bertipe `double`.
- `String getHargaFormatted()` => method khusus untuk menampilkan harga dalam format Rupiah yang mudah dibaca menggunakan `String.format("Rp %,0f", this.harga)`. Tanda `,` berfungsi sebagai pemisah ribuan (contoh: 250,000,000) dan `.0f` artinya tanpa angka desimal di belakang koma.

4.4. Class Utama

Kelas Utama berfungsi sebagai *entry point* (titik masuk) program. Karena diletakkan di luar package kendaraan, kelas ini memerlukan akses khusus untuk memanggil komponen di dalamnya.

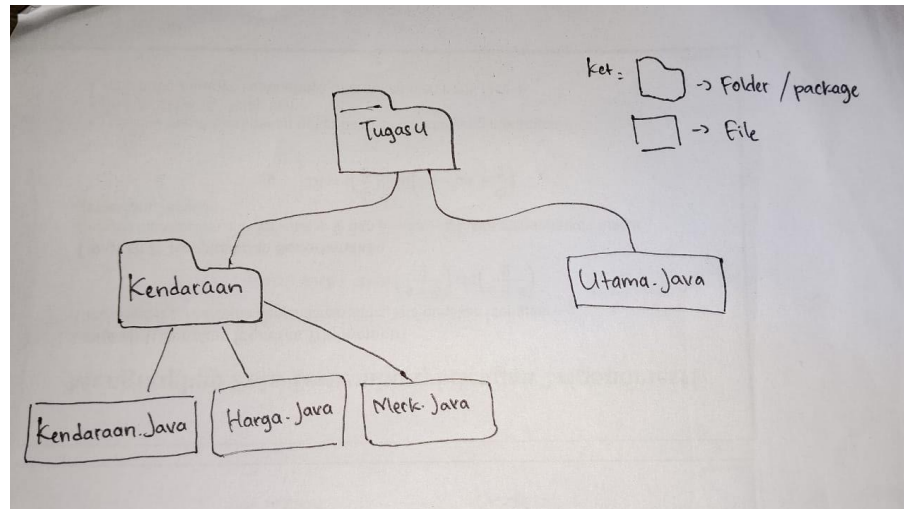
Penjelasan Kode Program :

- `import kendaraan.Harga` => karena kelas `Harga` berada di dalam package kendaraan, maka harus di-import terlebih dahulu agar dapat digunakan di kelas Utama yang berada di luar package tersebut.
- `public class Utama`, kelas ini tidak memiliki deklarasi package karena berada di luar folder kendaraan, dan berfungsi sebagai kelas utama yang menjalankan seluruh program.
- `public static void main(String[] args)` => method utama yang otomatis dijalankan pertama kali saat program dieksekusi oleh Java Virtual Machine (JVM).
- `Harga[] kendaraan = new Harga[3]` => membuat array bertipe `Harga` dengan kapasitas 3 elemen untuk menampung seluruh objek kendaraan. Penggunaan array ini membuat kode lebih efisien dibandingkan membuat 3 variabel terpisah.
- `kendaraan[0] = new Harga()` => membuat objek baru dari kelas konkrit `Harga` dan menyimpannya di indeks ke-0 array. Objek inilah yang merepresentasikan satu data kendaraan lengkap.
- `setJenisKendaraan()` => `setNamaMerk()`, `setHarga()`, memanggil method setter untuk mengisi data tiap kendaraan. Method-method ini dapat dipanggil karena `Harga` mewarisi seluruh method dari `Merk` dan `Kendaraan`.
- `for (int i = 0; i < kendaraan.length; i++)` => perulangan untuk menampilkan data semua kendaraan secara otomatis satu per satu tanpa harus menulis `println` berulang untuk setiap objek. `kendaraan.length` secara otomatis membaca jumlah elemen array yaitu 3.

- kendaraan[i].getHargaFormatted() => memanggil method khusus agar harga ditampilkan dalam format Rupiah yang rapi dan mudah dibaca.

4.4. Penejelasan flow program

Struktur Folder :



Gambar 4. 1 Struktur Program

Program ini disimpan dalam folder Tugas4 sebagai direktori utama. Di dalam folder Tugas4 terdapat satu folder/package bernama Kendaraan yang berisi tiga file yaitu Kendaraan.java, Merk.java, dan Harga.java. Selain folder Kendaraan, terdapat satu file bernama Utama.java yang berada langsung di dalam folder Tugas4 dan sengaja diletakkan di luar package Kendaraan sesuai ketentuan tugas. Struktur ini menerapkan konsep modularitas dalam pemrograman Java, di mana kelas-kelas yang saling berkaitan dikelompokkan dalam satu package, sementara kelas pengeksekusi utama berada di luar package tersebut.

Hierarki Pewarisan (Multi-level Inheritance):

Program ini mengimplementasikan konsep kelas abstrak dan hierarki pewarisan bertingkat (*multi-level inheritance*). Kelas Kendaraan berfungsi sebagai *superclass* paling atas yang bersifat abstrak, artinya kelas ini hanya mendefinisikan kerangka umum berupa atribut jenisKendaraan dan dua method abstrak tanpa implementasi, sehingga tidak bisa dibuat objeknya secara langsung. Kelas Merk kemudian menjadi *superclass sekaligus subclass* — mewarisi Kendaraan melalui keyword `extends` dan mengimplementasikan kedua method abstrak tersebut, sekaligus menambahkan atribut namaMerk beserta getter dan setter-nya. Kelas Harga berada di posisi paling bawah dalam hierarki sebagai *subclass* konkrit, melengkapi hierarki dengan menambahkan atribut `private double harga` beserta method `getHargaFormatted()` untuk memformat tampilan harga dalam format Rupiah.

Karena Harga mewarisi Merk yang mewarisi Kendaraan, kelas Harga secara tidak langsung mewarisi seluruh atribut dan method dari kedua kelas di atasnya.

Alur Eksekusi Program :

Seluruh logika program dijalankan melalui kelas Utama yang berada di luar package kendaraan, sehingga memerlukan perintah import kendaraan.Harga agar dapat menggunakan kelas Harga. Di dalam method main, program membuat tiga objek dari kelas Harga dan mengisi data masing-masing menggunakan method setter yang mewarisi dari kelas Merk dan Kendaraan. Ketiga objek tersebut kemudian disimpan ke dalam sebuah array Harga[], lalu ditampilkan secara terstruktur ke layar menggunakan perulangan for tanpa pengulangan kode yang tidak perlu. Pendekatan ini membuktikan bahwa konsep pewarisan kelas abstrak bekerja dengan baik karena method yang didefinisikan di Kendaraan dan Merk berhasil diimplementasikan dan dipanggil melalui objek kelas Harga, sekaligus menunjukkan penerapan enkapsulasi melalui penggunaan akses modifier private pada atribut harga.

5. Penjelasan Cara Runing Program

Untuk menjalankan program, langkah pertama yang dilakukan adalah navigasi, yaitu masuk ke folder tempat kode disimpan melalui CMD menggunakan perintah `cd path\ke\Tugas4`. Perintah ini wajib dilakukan agar CMD mengetahui lokasi file program yang akan dikompilasi dan dijalankan.

Setelah berada di dalam folder Tugas4, langkah selanjutnya adalah kompilasi, yaitu menerjemahkan kode Java menjadi file .class yang dapat dimengerti oleh komputer. Terdapat dua cara kompilasi yang dapat dilakukan:

- Cara pertama adalah kompilasi satu persatu, yaitu mengompilasi setiap file secara berurutan mulai dari Kendaraan.java, kemudian Merk.java, lalu Harga.java, dan terakhir Utama.java menggunakan perintah `javac` di setiap filenya. Urutan ini penting karena Merk bergantung pada Kendaraan dan Harga bergantung pada Merk, sehingga cara ini lebih direkomendasikan saat belajar karena membantu memahami ketergantungan antar kelas dan memudahkan pencarian error jika terjadi kesalahan pada file tertentu.
- Cara kedua adalah kompilasi sekaligus menggunakan perintah `javac Kendaraan/*.java Utama.java`, di mana `Kendaraan/*.java` berarti mengompilasi semua file .java dalam folder Kendaraan sekaligus dalam satu perintah. Cara ini lebih cepat dan efisien namun jika terjadi error, semua pesan error dari berbagai file akan muncul bersamaan sehingga lebih sulit untuk dilacak.

Setelah proses kompilasi selesai tanpa error, program dieksekusi menggunakan perintah `java Utama`. Perintah ini menjalankan kelas Utama yang di dalamnya terdapat method main sebagai titik masuk eksekusi program dan menghasilkan output berupa DATA KENDARAAN yang menampilkan data tiga objek kendaraan dengan informasi yang terstruktur. Setiap kendaraan menampilkan tiga atribut yaitu

Jenis yang berisi jenis kendaraan seperti Mobil, Motor, atau Sepeda, Merk yang berisi nama merek kendaraan seperti Toyota, Honda, atau Polygon, serta Harga yang menampilkan harga kendaraan dalam format Rupiah menggunakan pemisah ribuan melalui method `getHargaFormatted()`.

Program ini mendemonstrasikan penerapan konsep Pemrograman Berorientasi Objek (PBO) dalam Java, khususnya penggunaan kelas abstrak, package, dan enkapsulasi data. Pemisahan file kelas ke dalam package kendaraan menunjukkan penerapan modularitas kode yang baik, di mana kelas Kendaraan berfungsi sebagai kerangka abstrak utama, kelas Merk untuk mengelola informasi merek, dan kelas Harga untuk mengelola informasi harga kendaraan sekaligus menjadi satu-satunya kelas konkrit yang dapat dibuat objeknya secara langsung.